

First important step

As the size of the font contained inside terminals is quite small by standard, the professor provided us with a script to increase it. It must be run every time we restart the Kathara virtual machine.

Here is the code for `resize_xterm.sh`:

```
xrdb -merge .Xresources
```

It can be run with:

```
./resize_xterm.sh
```

Configuring the environment

Obviously, before starting the lab, we need to configure the parameters of the lab. This can be done in the `lab.conf` file, which can be created inside the directory for the laboratory.

Inside this file, we can configure interfaces and the respective network they're connected to.

This is an example:

```
r1[0]='A'
r1[bridged]=true
r1[sysctl]="net.ipv6.conf.all.forwarding=1"
r1[sysctl]="net.ipv4.ip_forward=1"

pc1[0]="A"
pc2[0]="A"
pc3[0]="A"
```

Basically, we're saying that the network to which the `eth0` interface of device `r1` is `A`. This also applies to `pc1`, `pc2` and `pc3`.

For `r1`, we're also configuring some extra parameters, like allowing the forwarding of packets with the IPv6 protocol, and the same with IPv4. This is done through the `sysctl` command.

Also, remember to allow through `kathara settings` some options. Specifically:

- enable IPv6 through option 9.
- modify the default network driver through option 10, selecting Kathara with Linux bridges.

Finally, we need to create the `device.startup` file in the environment. Device could be `pc1` , but also `r1` , and so on.

Assigning IP addresses to interfaces

Once the lab file has been configured, we need to assign IP addresses to the interfaces we've defined in `lab.conf` .

Addresses can be assigned in different ways. We can distinguish between:

- static addressing
- dynamic addressing

For static addressing, we can assign addresses in three main ways:

- Through the interfaces file.
- Using `ip` commands.
- Using `ifconfig` command

Let's analyze the first method. For it, we need to create the directory for the device, like `pc1/etc/network` . Inside it, we place the `interfaces` file. It could be something like this:

```
auto eth0
iface eth0 inet static
    address 192.168.100.25/29
    gateway 192.168.100.30
    dns-nameservers 8.8.8.8
```

With `address` , we're specifying the address we want to assign to the interface. The `gateway` , instead, is used to route all packets that do not match any routing table entries to that address. The `dns-nameservers` is used to define DNS servers.

Once we've defined the interface in this way, we need to make it work by adding this command in the startup file:

```
ip addr flush eth0  
ifup eth0
```

The second method is given by manual configuration through the `ip` commands. This can be done inside the startup files, like in this way:

```
ip addr flush eth0  
ip addr add 192.168.100.26/29 dev eth0
```

The third is given by the use of the `ifconfig` commands, which can be inserted inside the startup file as well.

```
ip addr flush eth0  
ifconfig eth0 192.168.100.27 netmask 255.255.255.248 up
```

Once we've done this, we've defined the correct static addresses.

Configuring routes

We've seen with `pc1` how to configure, in the interfaces file, the default gateway. To allow communication through devices which are not directly connected, we need to define some rules of routing. This especially for routers, who's task is to take packets and redirect them to the correct interface depending on the rules.

This concept applies, as well, to default gateways. If we want to send a packet outside the network, we have to send it to the default gateway router, who's task is to communicate with the rest of the internet.

For `pc2`, we need to add an option to allow the routing to the default gateway. This can be done with the `ip` command, inside the `pc2.startup`.

```
ip route add default via 192.168.100.30
```

Where `192.168.100.30` is the address of the interface `eth0` of `r1`, connected to the lan. The router will be responsible of redirecting packets depending on his rules.

For `pc3`, we can add inside the startup file the following command:

```
route add default gw 192.168.100.30
```

To add a route that is not a default one, we can use the following command:

```
ip route add 192.168.100.29/24 via 192.168.100.25 # example
```

Have particular care in configuring routes. When asked, each device should reach any other device. This can be done by setting up routes in all routers. Remember that a packet should be able to go back as well (when dealing with IP).

So, for example, if we have a route to go to direction 1 to 2, we'll have to have also from 2 to 1.

Configuring DNS nameservers

We've seen how, with `pc1`, we've defined the `dns-nameserver`, this allows to have connectivity with the rest of the internet.

If we want to implement this for each machine, this will have to be enforced by adding an option inside the `/etc/resolv.conf` file. This can be done inside startup files, in this way:

```
echo 'nameserver 8.8.8.8' > /etc/resolv.conf
```

Testing connectivity

Once the lab has been started, we can test connectivity between devices using the `ping` command, giving as an argument the IP of the device we're trying to connect to.

To check if a device has the IP assigned correctly, we can use the following command:

```
ifconfig
```

An example (from `pc1`):

```
ping 192.168.100.26 -c 4
```

We're defining that we're sending, with `-c`, 4 pings to that address,

We can also test connectivity to the internet by trying the following command:

```
wget google.com
```

Configuring IP addresses dynamically (DHCP)

Alongside configuring address statically, which could take some time, we can also configure addresses dynamically.

To do this, we need to configure a device as a DHCP server, which will handle the distribution of IP addresses to other devices configured as DHCP clients.

As for the DHCP server, we need to install `udhcpd` inside the machine, done in the startup file:

```
apt install udhcpd
```

Then, we need to define a configuration file for the server. This file is given by the `/etc/udhcpd.conf` file. Recommendation: create the directory for the device, then create inside it the etc directory, and then the configuration file.

```
start 192.168.100.18
end 192.168.100.29

max_leases 12

opt dns 8.8.8.8
option subnet 255.255.255.240
opt router 192.168.100.30
option domain pnd.test
option lease 600
```

With `start` we define the beginning of the range of IP addresses the server can distribute, while with `end` we define the last address.

Others are options (*which I'm not sure that actually needs to be configured*).

Then, to start the daemon, we insert the following lines inside the startup file:

```
dkpg -i /var/cache/apt/archives*.deb
```

```
apt install -f udhcp  
  
udhpc -f /etc/udhcpd.conf
```

To configure a machine as a client, we use the `dhclient` command.

```
dhclient eth0
```

Capturing Traffic

Packets flow continuously across the network, and to capture them, you can use network traffic dump tools such as `dumpcap`, `wireshark` / `tshark`, or `tcpdump`. All of these tools are based on the `pcap` library (or `winpcap` in Windows) and can visualize and save the captured data. Furthermore, tools like Wireshark and `tcpdump` are capable of analyzing and decoding the captured packets.

In order to save the traffic capturing externally from the Kathara machines, the Professor provided us with a script. It works by adding two virtual interfaces, `veth0` and `veth1`, connecting one to the virtual bridge, assigning an IP address to the other, and enabling them both. In this way, we can type the following:

```
sudo wireshark
```

and get access to the monitoring of all the machine interfaces, including virtual ones.

This is the `connect-lab.sh` script, for future usage:

```
if [ $# -eq 0 ]  
then  
    echo "ERROR: usage $0 IPADDR/NETMASK [NETWORK_NAME]"  
    exit 0  
fi  
echo "setting up host veth"  
  
sudo ip link del veth1 type veth  
sudo ip link add veth1 type veth  
sudo ip addr flush veth0  
sudo ip addr add $1 dev veth0  
if [ -z "$2" ]  
then
```

```

    echo "Network name not provided: assume only one lan
exists..."
    bridge="$(sudo docker network ls | grep -e 'kathara_.*' | grep
-o '^[a-z0-9]*' | head -n 1)"
else
    bridge="$(sudo docker network ls | grep -e
"kathara_${USER}.*_$2" | grep -o '^[a-z0-9]* ')"
fi

echo
echo "attaching veth1 to lan $2, bridge: kt-${bridge}"

sudo ip link set veth1 master kt-${bridge}

sudo ip link set veth0 up
sudo ip link set veth1 up
ip addr show dev veth1
ip addr show dev veth0

```

Another way to capture traffic is by using the `tcpdump` command inside the machine.

Specifically, one could specify:

```
tcpdump -w /shared/cap1.pcap
```

The `/shared` directory is a particular kind of directory, which can be accessed from outside the laboratory as well. It can be created, alongside other files, in the directory folder.

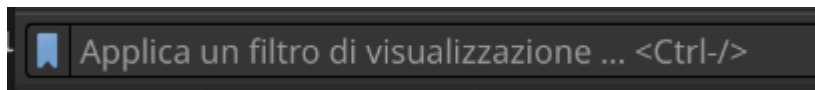
Analyzing traffic

Once we've captured traffic through `tcpdump`, or setup the virtual interface with the script provided above, we can analyze the packets that have been transmitted.

One major concern we may have when first taking a look at the screen is the amount of packages that have been tracked.

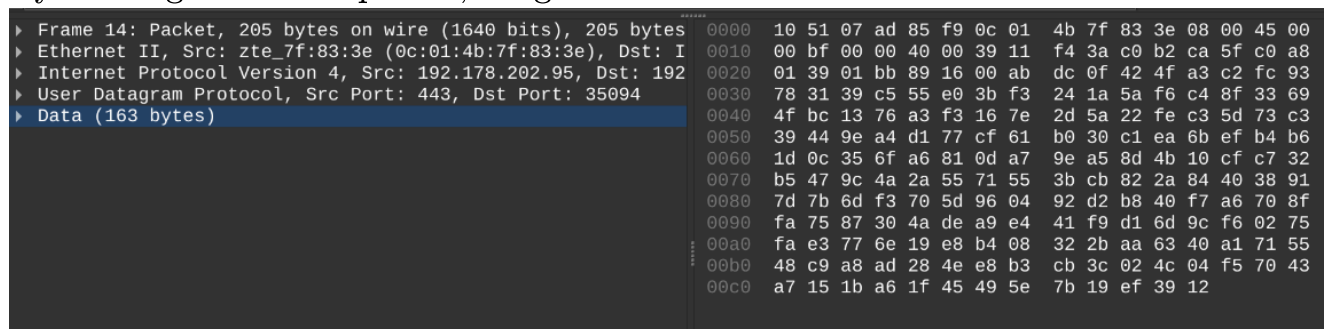
Luckily, Wireshark allows to apply filters to redirect our attention only to desired packets. This can be done essentially in two ways:

- Visual filters: applied in the search bar (photo below). Here we can specify, for example, the kind of protocol we want to focus on.



- Capture filters: they effectively reduce the amount of packets traced by only saving the content of the desired ones. To apply this kind of filters, we can use Berkeley Packet Filter (BPF) syntax. It's basically structured in this way:
protocol direction type . For the protocol field we may use tcp , udp , ip , while for the direction either src or dst , and for type we could use host , net . Also, filters can be concatenated by using logical operands, like && , || and ! .

By clicking once on a packet, we get access to this:



On the left, the different protocols used for the packet are specified, with the relative encapsulation. We can see how, for example, at transportation level the udp protocol is used, while at network level we have IP . If we click on the arrows, we can even get access to more information about that specific header of the network stack.

On the right, we have the encoding of the packet using hex values.

Also, by clicking on a packet with a right click, we get access to more options, like the "follow", which effectively allows us to see the flow of packets related to that one.

IPv6

IPv6, in contrast to IPv4, does not simply use one single address for a NIC, rather we can assign multiple addresses on a device.

Specifically, we can distinguish between a Link-Local address, assigned automatically to the machine, having a value typically starting with fe80:: , used to communicate with devices sharing the network link, and the Global Unicast Address (GUA), used to obtain global connectivity and routing.

Configuring IPv6 addresses

The ways we can follow to configure an IPv6 address statically are pretty much equivalent to the ones seen for IPv4, with some slight modifications.

First thing first, the address we assign for the gateway router must **not** be the GUA of the gateway, rather its Link-Local address.

Now, to assign addresses we can do this in the following ways:

1. Assign the address through the `interfaces` file.

The syntax is similar to IPv4, however we should use `inet6` instead of `inet`.

```
auto eth0
iface eth0 inet6 static
    address 2001:db8:1::10/64
    gateway fe80::1
```

2. Through `ip` command.

```
ip -6 addr add 2001:db8:1::11/64 dev eth0
# for default route
ip -6 route add default via fe80::1 dev eth0
```

Notice how we've specified the interface also when adding the route, this has to be done as it's necessary to avoid ambiguity through different interfaces connected to different networks, as multiple networks might have the exact same Link-Local address, and so the device must know through which port they must redirect traffic to.

This is also true for the `ping6` command. In fact, when trying to ping a Link-Local address, we'll have to specify the interface through which we want to perform this action.

3. Through `ifconfig` and `route` commands.

Inside `dev.startup` file, insert the following:

```
ifconfig eth0 inet6 add 2001:db8:2::12/64
# gateway
route -A inet6 add default gw fe80::1 eth0
```

One extra note: to allow routers to forward packets, we must insert the following command in the startup file of the router:

```
sysctl -w net.ipv6.conf.all.forwarding=1
```

Dynamic addressing in IPv6

By default, in Unix, addresses are assigned dynamically through the use of the Stateless Address Autoconfiguration (SLAAC) protocol. To manipulate how these addresses are generated, one must use the `sysctl` command, interacting with the Kernel.

One important notice: by default, Kathara does not allow the use of `sysctl` commands, and we need to open the lab in privileged mode. This can be done either by typing `kathara lstart --privileged`, or by using a shell file (like `lstart.sh`, provided in some labs). Notice that the former won't open the terminals of the labs automatically, so you'll have to open them yourself by typing: `kathara connect [dev_name]`.

One of the first parameters we can modify through `sysctl` is the `add_gen_mode`. This basically is used to tell how the IP will be generated. The possible values are:

- `0`, which allows to generate the address basing on EUI64 (default behaviour).
- `1`, does not generate a Link-Local, use EUI64 only for addresses generated by the autoconfiguration.
- `2`, generate stable private addresses, using the parameter given by `stable_secret`.
- `3`, generate stable private addresses using a random stable secret, if not set.

Here is an example inside a startup file:

```
sysctl -w net.ipv6.conf.eth0.addr_gen_mode=2
# we also need to add the stable secret for mode 2
sysctl -w net.ipv6.conf.eth0.stable_secret="fe80::1" # it's an
example
```

Another parameter is given by `use_tempaddr`. It is used to set the preference for Privacy Extensions, which have been introduced due to the fact standard SLAAC IP addresses were generated using the MAC address of the NIC, revealing sensitive information. Here are the possible values:

- `<=0`, disables Privacy Extensions.

- `==1` , enables Privacy Extensions, but the system will use public addresses rather than using temporary addresses.
- `>1` , enables Privacy Extensions and the system prefers using temporary addresses rather than public ones.

Example to maximise privacy:

```
sysctl -w net.ipv6.conf.eth0.use_tempaddr=2
sysctl -w net.ipv6.conf.eth0.temp_valid_lft=60 # 60 sec validity
sysctl -w net.ipv6.conf.eth0.temp_preferred_lft=30 # pref. time
```

Some extra parameters for temporary IP addresses can be defined with `temp_valid_lft` and `temp_preferred_lft`.

For additional info on `sysctl` commands, check out:

<https://www.kernel.org/doc/Documentation/networking/ip-sysctl.txt>.

Router Advertisement and Stateless DHCPv6

To make it possible for SLAAC to work, the router must send some special kind of message, the Router Advertisement, to advertise the network.

While, in a mixed setup given by SLAAC and Stateless DHCPv6, the router will handle the distribution of the IP prefix, and the DHCPv6 will distribute additional information like DNS servers.

To allow the router to send Router Advertisements, we should use the daemon `radvd`.

We'll have to create the `/etc/radvd.conf` file (which can be done, as we should remember, before starting the lab, simply by creating the `[dev]` folder, and putting it inside), and then insert something like this:

```
interface eth0 {
    AdvSendAvert on;

    AdvOtherConfigFlag on; # stateless directive

    prefix 2001:db8:1::/64 {
        AdvOnLink on;
        AdvAutonomous on; # SLAAC for IP
```

```
};  
};
```

To configure a router as a DNS server, we'll use instead the `dnsmasq` daemon. We'll have to create the `/etc/dnsmasq.conf` file.

```
interface=eth0 # configure correct interface  
  
dhcp-option=option6:dns-server,[2001:db8:1::1] # sends the IP of  
the  
# DNS server, typically the router itself  
  
dhcp-range=::,constructor:eth0,ra-stateless # configure range,  
# which is null in Stateless configuration, as DHCPv6 server only  
# provides options
```

To start both daemons, we can do that in the startup file or in the labs directly. Here are the commands:

```
radvd -C /etc/radvd.conf  
dnsmasq
```

We'll also have to configure clients to accept Router Advertisements message, and act as DHCPv6 clients.

```
sysctl -w net.ipv6.conf.eth0.accept_ra=1  
  
# start DHCPv6 client in background  
dhclient -6 -S eth0 # S flag used to indicate stateless
```

We can force the sending of a Router Solicitation message (for interface 0) through `rdisc6 eth0`.

DHCPv6 through Dibbler

We can either use the already seen `dhclient`, but also `dibbler`. We'll need to install it through our startup files first:

```
dpkg -i /shared/dibbler-client*.deb
```

Then, we'll have to create the configuration file `/etc/dibbler/client.conf`:

```
log-level 7
iface eth0 {
    stateless
}
```

Then start the daemon through:

```
dibbler-client start
```

DHCPv6 Prefix Delegation

While dealing with Prefix Delegation, our DHCP router will function as a client for a ISP, by which he will receive a bunch of prefixes he will be able to distribute.

To configure this, we'll use `dibbler`, allowing to work with prefix delegation:

```
log-level 8
iface eth0 {
    pd
}
```

MTU Management

We can decide the maximum transfer dimension for packets within a certain link. This can be done with the following command:

```
ip link set dev eth0 mtu 1280 #example
```

It will need to be applied at all ends of a specific network link, in order to be operational.

We can test out the receive of a "Packet Too Big" message by forcing the MTU through the `ping` command, in this way:

```
ping -6 -s 1500 -M do 2001:db8:cafe:4::2
# with -M do we force the packet not to be fragmented
```

Additionally, we can trace the MTU among the different links we pass through by using the `tracpath` command:

```
tracpath -6 2001:db8:cafe:4::2
```